

02285 AI and MAS, F26

Domain-independent heuristics

Curriculum for week 3: Section 11.3 in R&N, Sections 2.5 and 2.7 in Geffner & Bonet.

Heuristics from relaxed problems

Recall: A **relaxed problem** P' is a simplified version of the original problem P with fewer restrictions. Computing the **actual cost** $h_{P'}^*(s_0)$ of the relaxed problem always gives an admissible heuristic for the original problem (so we set $h(s) = h_{P'}^*(s)$).

Key development in modern planning research: Domain-independent heuristics obtained from **automatic** problem relaxations.

Example. E.g. using the **goal count** heuristic from last week, where the relaxation P' is to assume each subgoal can be reached in one action:

$$h_{gc}(s) = h_{P'}^*(s) = \text{number of goal literals not satisfied in } s.$$

With $P = \text{simplified gripper problem}$ we have $h_{P'}^*(s) = h_P^*$ (optimality).

Is assuming that each subgoal can be reached in one action a relaxation of any planning problem? No. Some actions might achieve several goal literals. E.g. a block-pushing agent that can push two blocks at a time.

What are relevant problem relaxations in the hospital domain? E.g.: ignore walls; ignore that cells has to be free; ignore that objects can't be in more than one cell (see delete-relaxation later).

Ignore preconditions heuristic

In the following we introduce several heuristics induced by relaxed problems.

Definition (Ignore preconditions heuristic)

Relax problem by ignoring all preconditions. Thus every action becomes applicable in every state.

ACTION: $MoveAB(x)$

PRECONDITION: $\cancel{Box(x)} \wedge \cancel{In(x, A)}$

EFFECT: $In(x, B) \wedge \neg In(x, A)$

ACTION: $Buy(x, y)$

PRECONDITION: $\cancel{Buyable(x)} \wedge \cancel{Place(y)} \wedge \cancel{At(y)} \wedge \cancel{Sells(y, x)}$

EFFECT: $Have(x)$

How will this heuristic work on simplified Gripper problem? Quite bad.
Calculating it takes just as long as solving the original problem.

And what about the milk-and-bananas problem? A bit better, but a rather expensive way to calculate the goal count heuristic.

Ignore preconditions and non-goal literals heuristic

Definition (Ignore preconditions and non-goal literals heuristic h_{ip})

Relax by ignoring all preconditions **and** all effect literals except those occurring in the goal.

ACTION: $MoveAB(x)$

PRECONDITION: $\cancel{Box(x)} \wedge \cancel{In(x, A)}$

EFFECT: $In(x, B) \wedge \cancel{In(x, A)}$

Do we have $h_{ip} = h_{gc}$, that is, is the new heuristic equivalent to the goal count heuristic that simply counts the number of unsatisfied goal literals?

No. Some actions can achieve several goal literals, and some actions might undo the effect of others.

h_{gc} was not admissible. Is h_{ip} ? Yes, it comes from a relaxed problem: Any solution to the original problem is also a solution when preconditions and non-goal literals are ignored.

Remark. Calculating $h_{ip}(s)$ is a **set-cover problem** (given set U and set of sets S , find the smallest subset $C \subseteq S$ s.t. $\bigcup C = U$). Set-covering is **NP-hard**, but a tractable greedy approximation exists.

Delete-relaxation heuristic

Definition (Delete-relaxation (or 'ignore delete list') heuristic h^+)

Relax problem by removing all negative literals from the effects of actions (equivalently: set all delete lists to $\emptyset$).

ACTION: $Go(x, y)$

PRECONDITION: $Place(x) \wedge Place(y) \wedge At(x)$

EFFECT: $At(y) \wedge \neg At(x)$

ACTION: $Eat(cake)$

PRECONDITION: $Have(cake)$

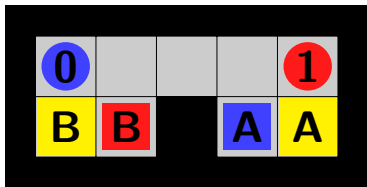
EFFECT: $Full \wedge \neg Have(cake)$

Only **admissible** if all *goals* and *preconditions* only contain positive literals, i.e., atoms. **Why?** A negative goal will never be found by h^+ .

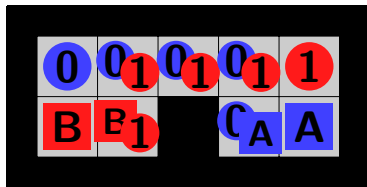
Easy to ensure: Any planning problem P can be translated into an equivalent planning problem without negative literals in these places.

How? Replace every negative literal $\neg A$ by a new positive literal $\bar{A}$.

Delete-relaxation heuristic example



Original level



Delete relaxation

Cost of relaxed solution: 4. So $h^+(s_0) = 4$.

Let $s_1 = \text{RESULT}(s_0, \text{Move}(0, E) | \text{Move}(1, W))$. Then $h^+(s_1) = 3$.

That's a bit misleading in this case, since it leads to a necessary conflict between the 2 agents.

Real cost $h^*(s_0) = 8$.

Also, unfortunately, calculating h^+ is **NP-hard**. We need more ideas...

Additive heuristic

Definition (Additive heuristic h_{add})

Relax problem by:

1. removing negative literals from effects (as in delete-relaxation).
2. assume **subgoal independence**: The cost of achieving a goal $g_1 \wedge g_2 \wedge \dots \wedge g_n$ is the sum of the costs of achieving each of the g_i .

Letting $h^*(g, s)$ denote the optimal cost of achieving g from s , the subgoal independence assumption amounts to assuming

$$h^*\left(\bigwedge_i g_i, s\right) = \sum_i h^*(g_i, s)$$

Find planning problem where the subgoal independence assumption is **strictly pessimistic**, i.e., $\sum_i h^*(g_i, s) > h^*(\bigwedge_i g_i, s)$. Achieve p and q , and there is one action that achieves both.

Find planning problem where the subgoal independence assumption is **strictly optimistic**, i.e., $\sum_i h^*(g_i, s) < h^*(\bigwedge_i g_i, s)$. Achieve p and q , there are actions a_p and a_q achieving them, but a_p also achieves $\neg r$ and r is precondition of a_q .

Additive heuristic (cont'd)

Since the subgoal independence assumption is sometimes pessimistic, the additive heuristic is **not admissible**.

But the good news is: it is **tractable** (meaning that can be computed in polynomial time in the size of the planning problem, i.e., in the sum of the sizes of the initial state, the sizes of the action schemas and the size of the goal formula).

Conventions for the remaining slides:

- All planning problems have only positive literals (atoms) in goals and preconditions (without loss of generality).
- We use symbols $p, p_1, p_2, \dots$ to denote ground atoms (e.g. $In(1, A)$ or $Have(Milk)$).
- Recall from last week that $ADD(a)$ denotes the set of positive literals in the effect of the action a . In a delete-relaxation, $ADD(a)$ contains **all** effects of a .

Additive heuristic (cont'd)

The additive heuristic h_{add} for a planning problem $P = (\mathcal{A}, s_0, g)$ can be expressed quite neatly using a recursive definition. It is non-trivial to compute, however.

Definition (Additive heuristic h_{add} , recursive definition)

$$h_{add}(s) = h_{add}(g, s) \quad (\text{Note the overloading of } h_{add})$$

$$h_{add}(\bigwedge_i p_i, s) = \sum_i h_{add}(p_i, s) \quad (\text{subgoal independence assumption})$$

$$h_{add}(p, s) = \begin{cases} 0 & \text{if } p \in s \\ \infty & \text{if } p \notin s \text{ and for every action } a, p \notin \text{ADD}(a) \\ 1 + \min\{h_{add}(\text{PRECOND}(a), s) \mid p \in \text{ADD}(a)\} & \text{otherwise} \end{cases}$$

Note how the subgoal independence assumption is treated by the second clause for h_{add} above, and delete-relaxation is treated in the last clause (we only consider ADD, not all effects).

Additive heuristic example

We consider the simplified Gripper problem from last week with action schemas

ACTION: *MoveAB*(x)

PRECONDITION: $\text{Box}(x) \wedge \text{In}(x, A)$

EFFECT: $\text{In}(x, B) \wedge \neg \text{In}(x, A)$

ACTION: *MoveBA*(x)

PRECONDITION: $\text{Box}(x) \wedge \text{In}(x, B)$

EFFECT: $\text{In}(x, A) \wedge \neg \text{In}(x, B)$

Init state is $s_0 = \bigwedge_{i=1, \dots, n} \text{In}(i, A)$ and goal is $g = \bigwedge_{i=1, \dots, n} \text{In}(i, B)$.

Then:

$$h_{\text{add}}(g, s_0) = h_{\text{add}}(\bigwedge_{i=1, \dots, n} \text{In}(i, B), s_0) = \sum_{i=1, \dots, n} h_{\text{add}}(\text{In}(i, B), s_0).$$

For each $i = 1, \dots, n$ we get:

$$\begin{aligned} h_{\text{add}}(\text{In}(i, B), s_0) &= 1 + \min\{h_{\text{add}}(\text{PRECOND}(a), s_0) \mid \text{In}(i, B) \in \text{ADD}(a)\} \\ &= 1 + h_{\text{add}}(\text{Box}(i) \wedge \text{In}(i, A), s_0) \\ &= 1 + h_{\text{add}}(\text{Box}(i), s_0) + h_{\text{add}}(\text{In}(i, A), s_0) \\ &= 1 + 0 + 0 = 1. \end{aligned}$$

Combining the above we get $h_{\text{add}}(g, s_0) = n$. Hence $h_{\text{add}} = h^*$.

Another additive heuristic example

ACTION: *Buy*(x, y) (buy item x at location y)

PRECONDITION: $Buyable(x) \wedge Place(y) \wedge At(y) \wedge Sells(y, x)$

EFFECT: $Have(x)$

ACTION: *Go*(x, y)

PRECONDITION: $Place(x) \wedge Place(y) \wedge At(x)$

EFFECT: $At(y) \wedge \neg At(x)$

$s_0 = Buyable(Milk) \wedge Buyable(Bananas) \wedge Buyable(Drill) \wedge Place(Home) \wedge Place(Netto) \wedge Place(Bilka) \wedge At(Home) \wedge Sells(Netto, Milk) \wedge Sells(Netto, Bananas) \wedge Sells(Bilka, Milk) \wedge Sells(Bilka, Bananas) \wedge Sells(Bilka, Drill)$.

$g = At(Home) \wedge Have(Milk) \wedge Have(Bananas) \wedge Have(Drill)$.

$$\begin{aligned} h_{add}(Have(M), s_0) &= 1 + \min\{h_{add}(Buyable(M) \wedge Place(N) \wedge At(N) \wedge Sells(N, M), s_0), \\ &\quad h_{add}(Buyable(M) \wedge Place(B) \wedge At(B) \wedge Sells(B, M), s_0), \\ &\quad h_{add}(Buyable(M) \wedge Place(H) \wedge At(H) \wedge Sells(H, M), s_0), \dots\} \\ &= 1 + \min\{1, 1, \infty, \dots\} = 2. \end{aligned}$$

$$\begin{aligned} h_{add}(s_0) &= h_{add}(g, s_0) \\ &= h_{add}(At(H), s_0) + h_{add}(Have(M), s_0) + h_{add}(Have(B), s_0) + h_{add}(Have(D), s_0) \\ &= 0 + 2 + 2 + 2 = 6 > h^*(s_0). \end{aligned}$$

Why is h_{add} not optimal here? Because subgoal independence fails.

HSP, Fast Forward and beyond

HSP (see the Geffner & Bonet) is a planner using WA^* or greedy-best first search with h_{add} . State-of-the-art in the late 1990s, solving 789 out of the 1150 problems in the International Planning Competition (IPC).

Most modern state-of-the-art planners are still based on the same underlying ideas, except further refined:

- **Search refinements:** e.g. 1) first an incomplete & fast hill climbing search followed by a complete & slow greedy best-first search. Used in the planner Fast Forward (FF). 2) Best-first search with multiple frontiers, used in Fast Downward (refinement of FF). 3) RWA^* (restarting WA^*) that performs iterated WA^* searches with decreasing values of W . Used in the LAMA planner.
- **Heuristic refinements:** e.g. adjustments to the additive heuristics to make it more precise. Used in Fast Forward (see later slides).

Prior to IPC 2023, the most efficient planner is Fast Downward Stone Soup, a portfolio-based planner that makes use of 144 different possible configurations of Fast Downward.

Max heuristic

Definition (Max heuristic h_{max})

Exactly like the additive heuristic except h_{add} is replaced by h_{max} and the sum clause

$$h_{add}(\bigwedge_i p_i, s) = \sum_i h_{add}(p_i, s)$$

is replaced by a maximum clause

$$h_{max}(\bigwedge_i p_i, s) = \max_i h_{max}(p_i, s).$$

This corresponds to a change in relaxation where subgoal independence is no longer assumed.

Is the new max heuristic admissible? Yes, it assumes that the cost of making a conjunction true is the cost of the most expensive conjunct.

In example from previous slide: $h_{max}(s_0) = \max\{h_{max}(\text{Have}(M), s_0), h_{max}(\text{Have}(B), s_0), h_{max}(\text{Have}(D), s_0)\} = 2.$

- **Additive heuristic:** Not admissible, but often very informative.
- **Max heuristic:** Admissible, but often not very informative.

Important about the additive and max heuristics

- The additive/max heuristic define a path-finding problem over the **atom space**: Each node is an atom rather than a state.
- It is, however, not an ordinary graph but a directed hypergraph: If p is a node (an atom) and a an action with $p \in \text{ADD}(a)$, then there is a hyperedge $(\text{PRECOND}(a), p)$ labelled by a in the graph (it is a hyperedge because $\text{PRECOND}(a)$ is a set of atoms, i.e., a set of nodes).
- The size of the hypergraph is linear in the number of ground atoms and actions, and hence the additive and max heuristics become *polynomial* in the number of ground atoms and actions.
- This is much better than calculating the entire state space which is *exponential* in the number of ground atoms and actions.

Decomposition heuristics

Let $P = (\mathcal{A}, s_0, g)$ be a planning problem with goal $g = g_1 \wedge g_2 \wedge \dots \wedge g_n$. Let $h_P^*(g_i, s)$ denote the optimal cost of reaching a state satisfying g_i from s . These are computed by finding optimal plans for g_i , e.g. using BFS or A^* with an admissible heuristic. Define:

$$h_1(s) = \max_i h_P^*(g_i, s) \quad (\text{a max heuristic})$$

$$h_2(s) = \sum_i h_P^*(g_i, s) \quad (\text{a sum heuristic})$$

Are h_1 and h_2 admissible? h_1 is, but not h_2 . But h_2 can easily be more precise and guide the search better.

How do these heuristics differ from the previous sum and additive heuristics? Here we still keep track of the negative effects, and we don't propagate the subgoal independence assumption.

By definition, h_2 is exact ($h_2(s) = h_P^*(g, s)$) when the subgoals are independent. h_1 is exact when the same is true and subgoals can be achieved in parallel (cf. MAPF).

Better heuristics: Planning graphs

Planning graphs have **two types of nodes**:

- **Action nodes.** An action from the planning problem or the *no-op* action (do nothing—also called *persistence action*).
- **Literal nodes.** A literal of the planning problem.

The nodes are divided into **levels**:

$$S_0, A_0, S_1, A_1, S_2, A_2, \dots, S_n, A_n$$

of two **alternating types**:

- **State levels** S_i : sets of literal nodes.
- **Action levels** A_i : sets of action nodes.

The first level, S_0 : the literals of the **start state** of the planning problem.

The construction of planning graphs

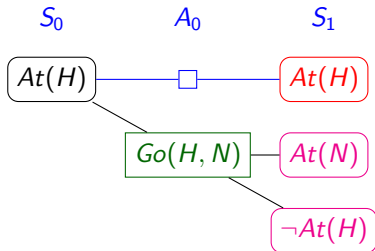
Given a state level S_i , $i \geq 0$, the subsequent action level A_i and subsequent state level S_{i+1} are defined as follows:

No-ops. For each node of S_i :

1. Make a copy of it in S_{i+1} .
2. Connect the node and its copy through a persistence action in A_i .

Actions. For each action a whose preconditions all occur at level S_i :

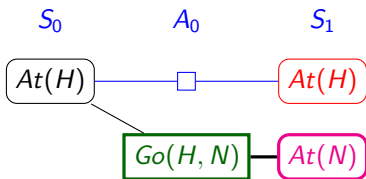
1. Add a to level A_i .
2. Add all effects of a to level S_{i+1} (except already existing ones).
3. Connect a to all its preconditions in S_i and all its effects in S_{i+1} .



The first automated planner that scaled reasonably well was GRAPHPLAN from 1995 based on building such planning graphs.

Planning graphs for Fast Forward

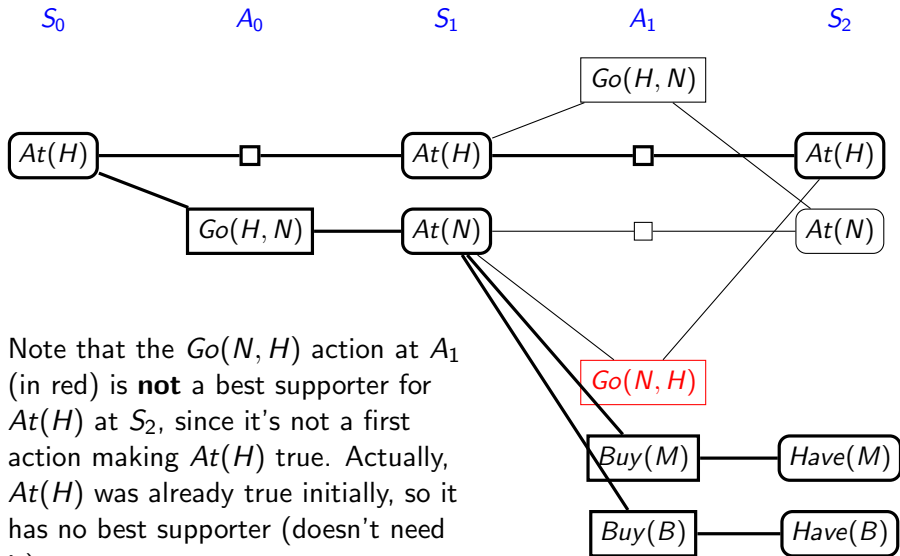
Build the same planning graph as before, but ignore the delete list.



Stop when building the first layer containing all goal literals. **When will this happen at latest?** After we have constructed as many layers as the problem has ground atoms. The heuristic h_{FF} for Fast Forward is then the cost of the (relaxed) solution extracted from this planning graph.

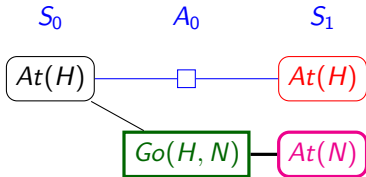
Best supporter for an atom A : An action a that was the first to add A to the planning graph. For instance $Go(H, N)$ is chosen as best supporter for $At(N)$ above (highlighted in boldface).

Best supporters example: milk-and-bananas



Note that the $Go(N, H)$ action at A_1 (in red) is **not** a best supporter for $At(H)$ at S_2 , since it's not a first action making $At(H)$ true. Actually, $At(H)$ was already true initially, so it has no best supporter (doesn't need it).

Planning graphs for Fast Forward (cont'd)



Algorithm for Fast Forward

Let G be set of goal atoms. Let m be smallest such that $G \subseteq S_m$. Let $G_m = G$. Then:

for $i = m - 1, \dots, 0$ **do**

$B_i \leftarrow \{a \in A_i \mid a \text{ is a best supporter of an atom in } G_{i+1}\}$

$G_i \leftarrow (G_{i+1} - \bigcup_{a \in B_i} \text{ADD}(a)) \cup \bigcup_{a \in B_i} \text{PRE}(a)$

The final relaxed solution consists of the actions $B_0, \dots, B_{m-1}$, where all actions in B_i have to be executed before all actions in B_{i+1} (but where actions inside each B_i can be executed in any order, cf. partial-order planning later in the course). h_{FF} = cost of this solution.

